

MICROPROCESSOR BASED SYSTEMS

CMP 410

LECTURE 3

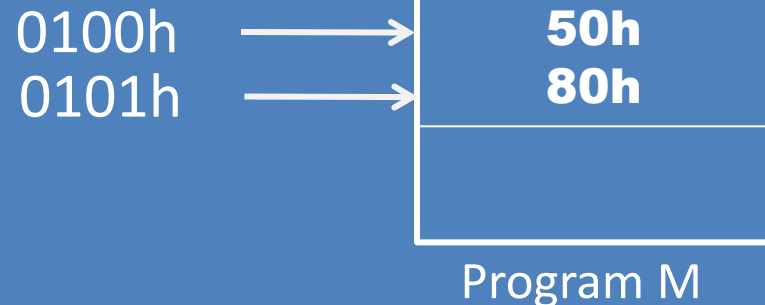
<https://AssemSite8.wix.com/site8>

DB

Directive used by assembler to store the permanent data into the program memory (**during the burning process**)

ORG 100h

DB 50h, 90h



The Stack

The **stack** is part of the RAM

In 8051, all RAM locations can be as stack

The Stack Pointer (SP)

The **SP** is one of the SFR

It has initial value "**07h**"

It points toward the top of the stack area

The Stack is affected by

1- **Push & Pop** instructions

2- **Call & Ret** instructions

3- **the interrupts** (automatic call)

4- **Reti** instructions (ret from interrupt)

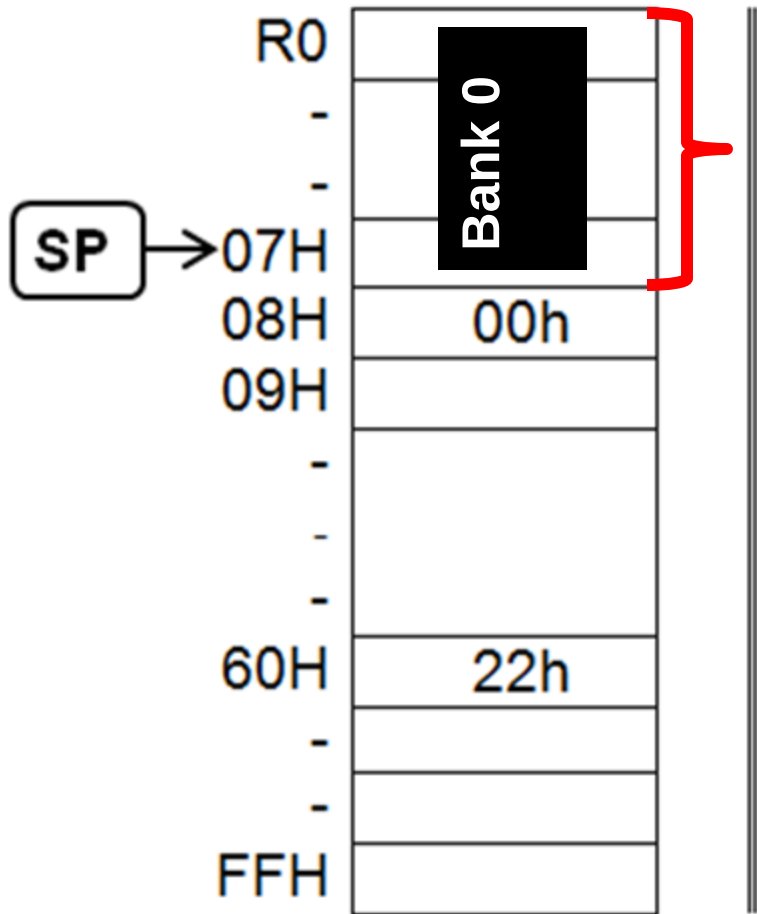
The data transfers from/to Stack

Last input is first output

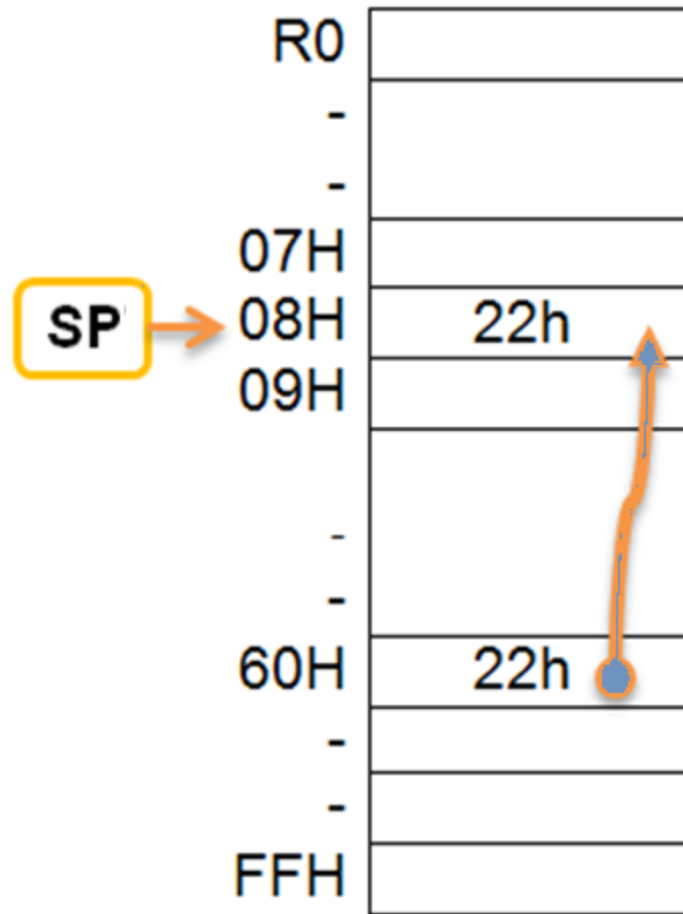
Pushing data-byte into the stack

PUSH 60h

Source address



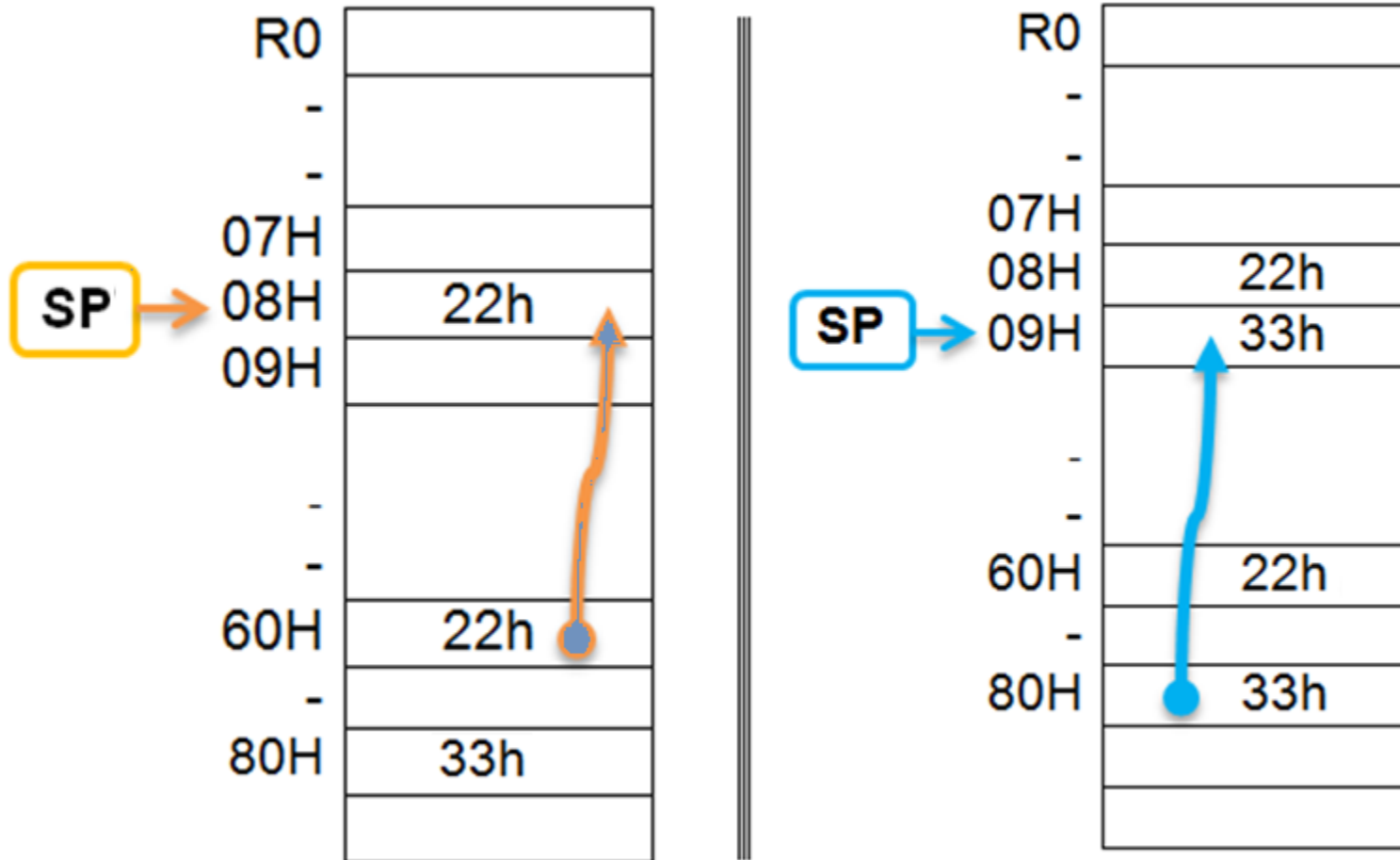
Initial values



After PUSH

Pushing other data-byte into the stack

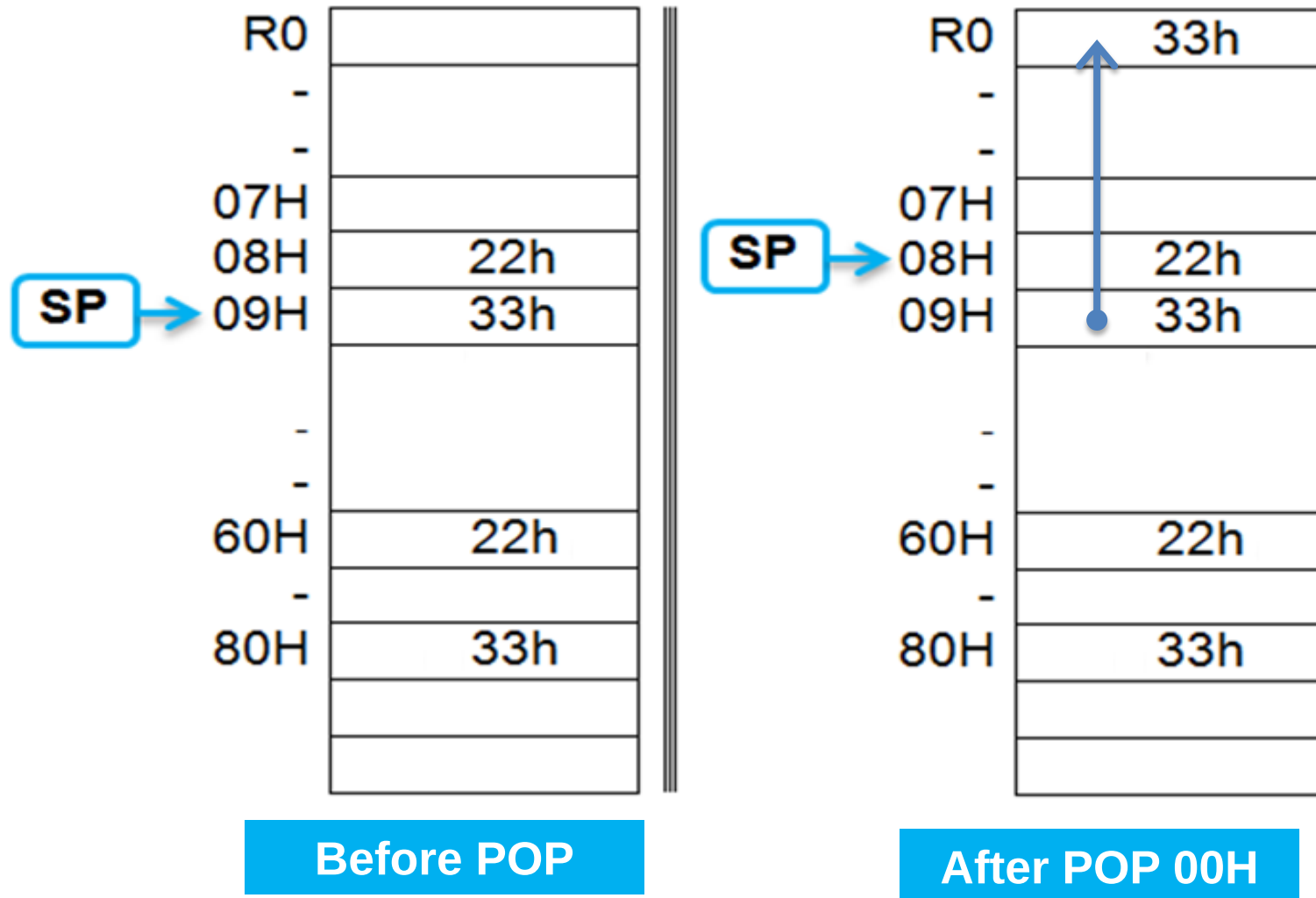
PUSH 80h



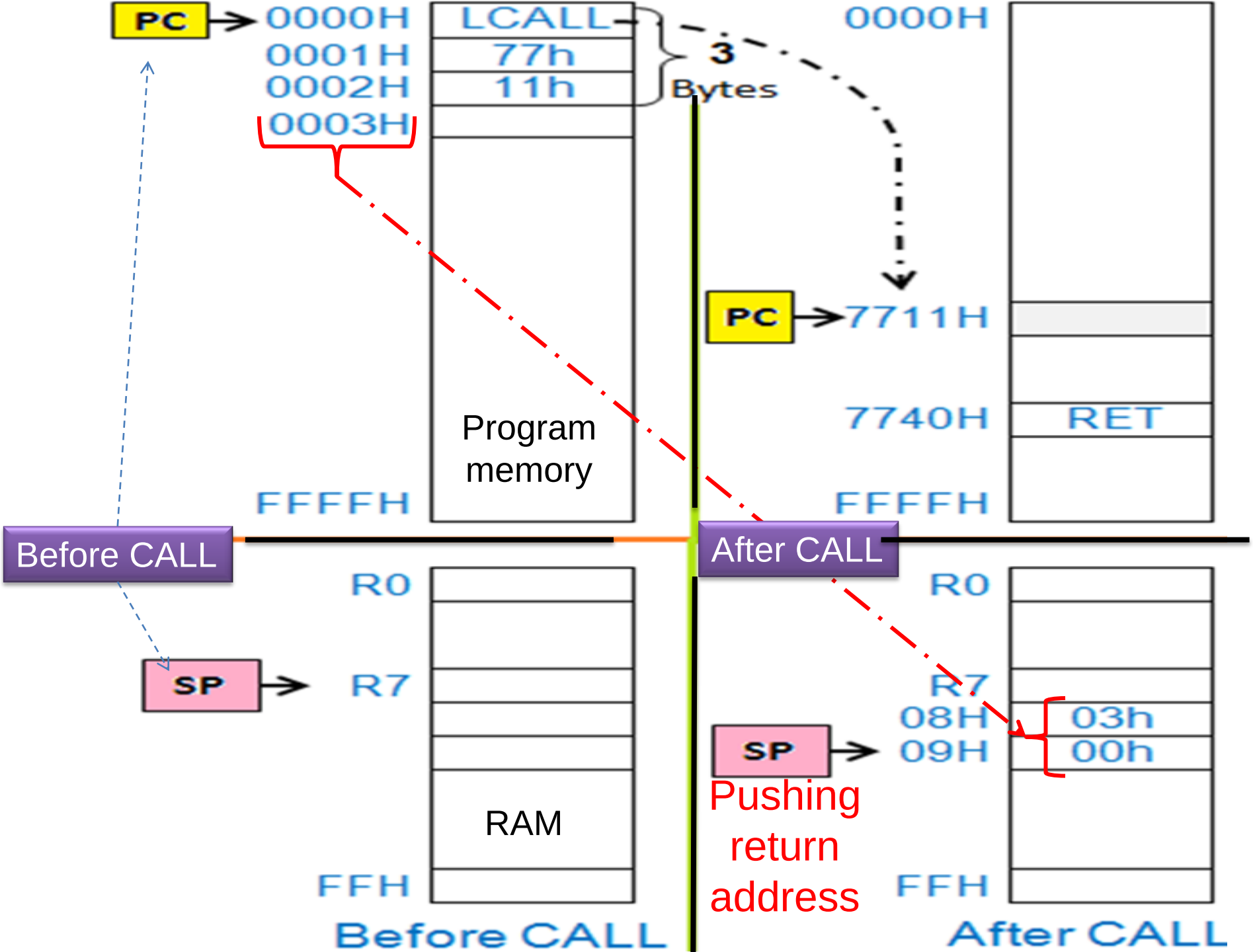
After PUSH 80h

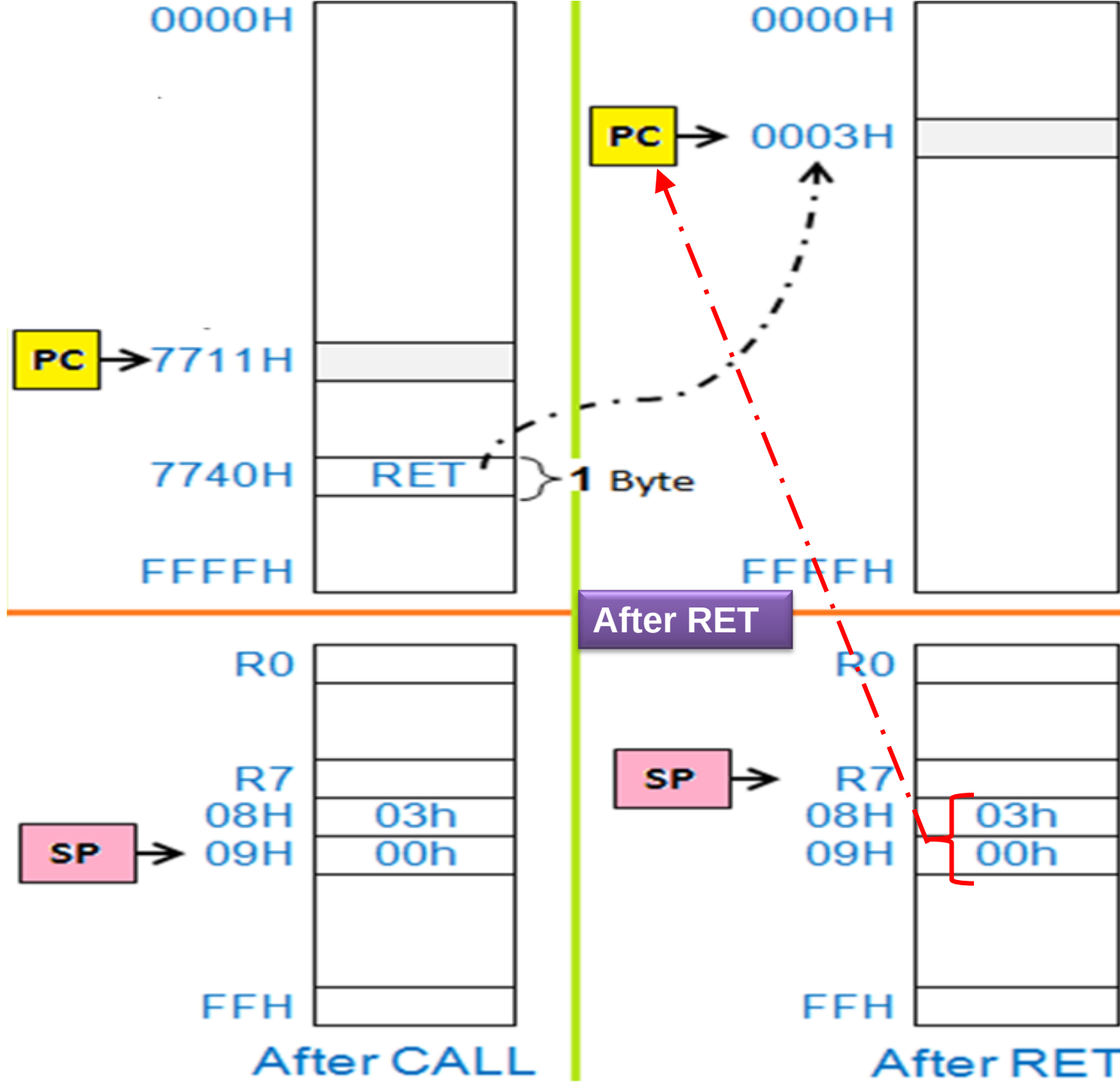
After Pop data-byte from the stack

POP 00H



**The execution of instruction
“LCALL” on the stack
(2 slides)**





The instruction types of MCS-51

The ISA of MCS-51 family includes **5 types** of instructions as following:

- a) **Data transfer** Instructions.
- b) **Logical operation** instructions.
- c) **Bit-Oriented** Instructions.
- d) **Arithmetic operation** Instructions.
- e) **Branching** Instructions.

some terminologies of ISA for MCS-51

Instruction's terminology	Description	Example
A	The Accumulator	CPL A
C	The Carry flag	CLR C
Bit	The addressable bit	SETB 00H
Rn	The Register R0 to R7 of the selected bank	DEC R5
Rx	Address of any internal RAM location (from 00h to FFh) or (from 0 to 255) or the SFR's names	MOV PSW, 20H
@Ri	Indirect 8-bit address of internal or external RAM location (i = 1 or 0 only)	INC @R0 DEC @R1



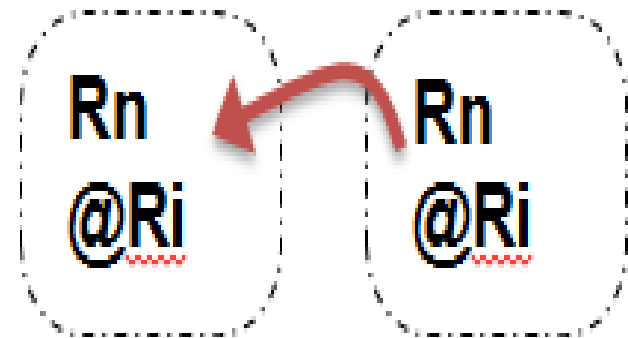
Data Transfer Instructions 1

Instruction	Description
MOV @Ri, #data	Move 8-bit data → internal RAM (indirectly[Ri])
MOV @Ri, A	Move data from A → internal RAM (indirectly[Ri])
MOV @Ri, Rx	Move data from internal RAM → internal RAM (indirectly[Ri])
MOV A, #data	Move 8-bit data → A
MOV A, @Ri	Move data from internal RAM (indirectly[Ri]) → A
MOV A, Rx	Move data from internal RAM → A
MOV A, Rn	Move data from (R0... R7) → A
MOV Rx, #data	Move 8-bit data → internal RAM
MOV Rx, @Ri	Move data from internal RAM (indirectly[Ri]) → internal RAM
MOV Rx, A	Move data from A → internal RAM
MOV Rx, Rx	Move data from internal RAM → internal RAM
MOV Rx, Rn	Move data from (R0... R7) → internal RAM
MOV DPTR, #data16	Move 16-bit data → DPTR
MOV Rn, #data	Move 8-bit data → (R0... R7)
MOV Rn, A	Move data from A → (R0... R7)
MOV Rn, Rx	Move data from internal RAM → (R0... R7)
MOVC A, @A+DPTR	Move data from Flash-M (indirectly [A+DPTR]) → A
MOVC A, @A+PC	Move data from Flash-M (indirectly [A+PC]) → A
MOVX A, @DPTR	Move data from external RAM (indirectly[DPTR]) → A
MOVX A, @Ri	Move data from external RAM (indirectly[Ri]) → A
MOVX @DPTR, A	Move data from A → external RAM (indirectly[DPTR])
MOVX @Ri, A	Move data from A → external RAM (indirectly[Ri])

Available MOV-instructions

	Destination operand	Source operand
MOV	Rx	Rx
	Rn	Rn
	@Ri	@Ri
	A	A
		#data

The available operands of "MOV"



Invalid data-movement instructions

MOV R1, 8h

R0	
R1	22h
R2	
R3	
R4	
R5	
R6	
R7	
8H	77h
9H	

Before execution "MOV R1, 8H"

R0	
R1	77h
R2	
R3	
R4	
R5	
R6	
R7	
8H	77h
9H	

After execution "MOV R1, 8H"

MOV P3, A

-	
-	
-	
P3 = B0H	8Dh
-	
-	
A = E0H	55h

Before execution "MOV P3, A"

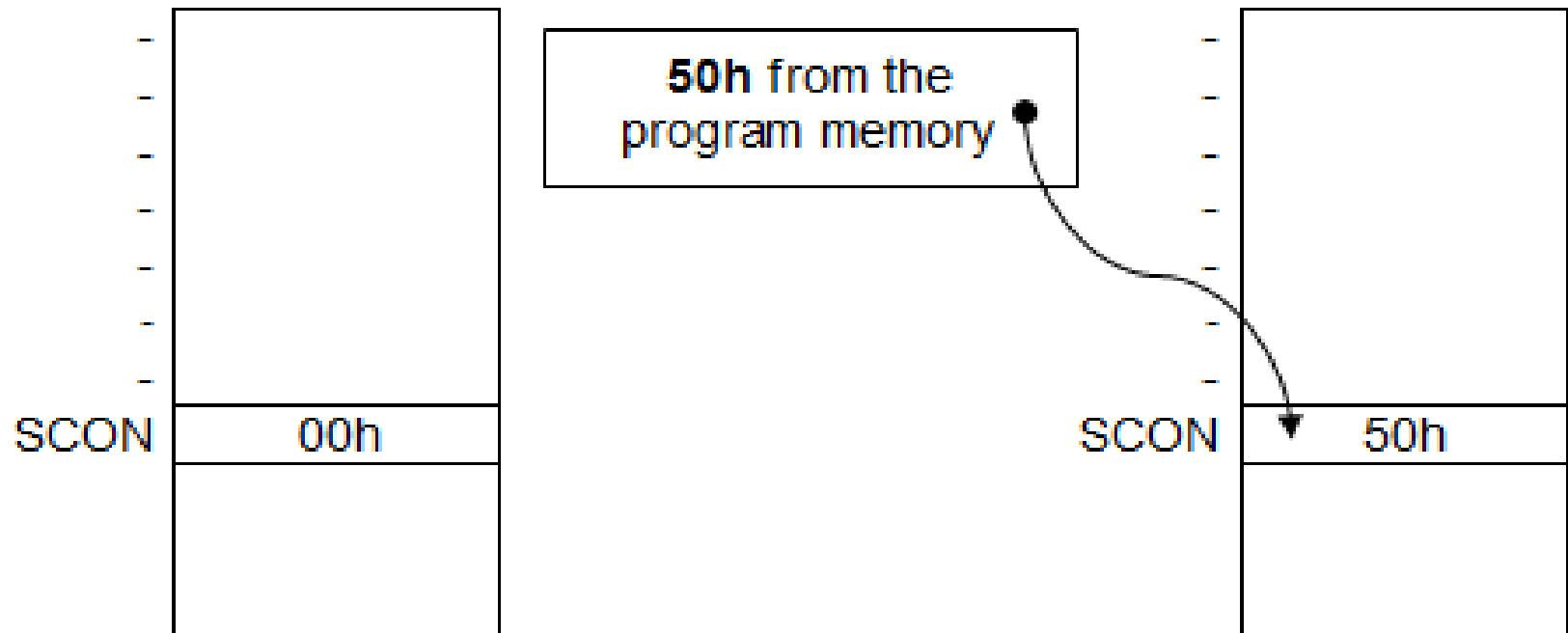
10/16/2018

-	
-	
-	
P3 = B0H	55h
-	
-	
A = E0H	55h

After execution "MOV P3, A"

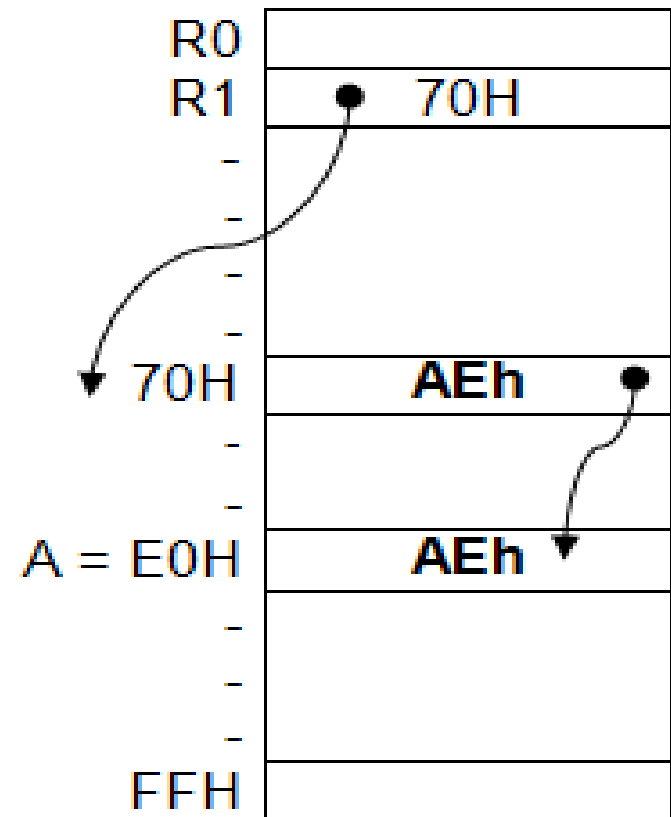
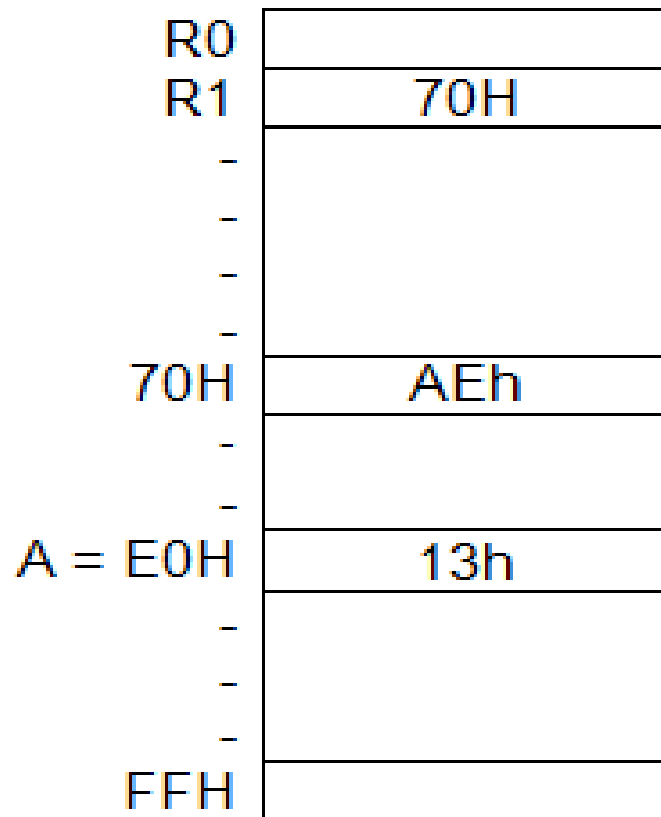
18

MOV SCON, #50h



Before and after the execution of the instruction “MOV SCON, #50H”

MOV A, @R1



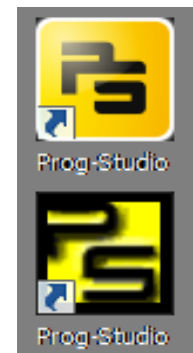
Before and after the execution “MOV A, @R1”

EXAMPLE

Draw Memory block-diagram and write assembly sub-program to read 2 samples (digital data) from a digital sensor via port 2 and store them into 2 successive internal RAM locations starting from address 77h using indirect addressing @Ri of MCS-51.

The subprogram

Using simulator “prog-studio”



```
INCLUDE 8051.MC
```

```
MOV P2,#FFH ; Setting port 2 as input
```

```
MOV R0,#77H ; Loading R0 by 77h
```

→

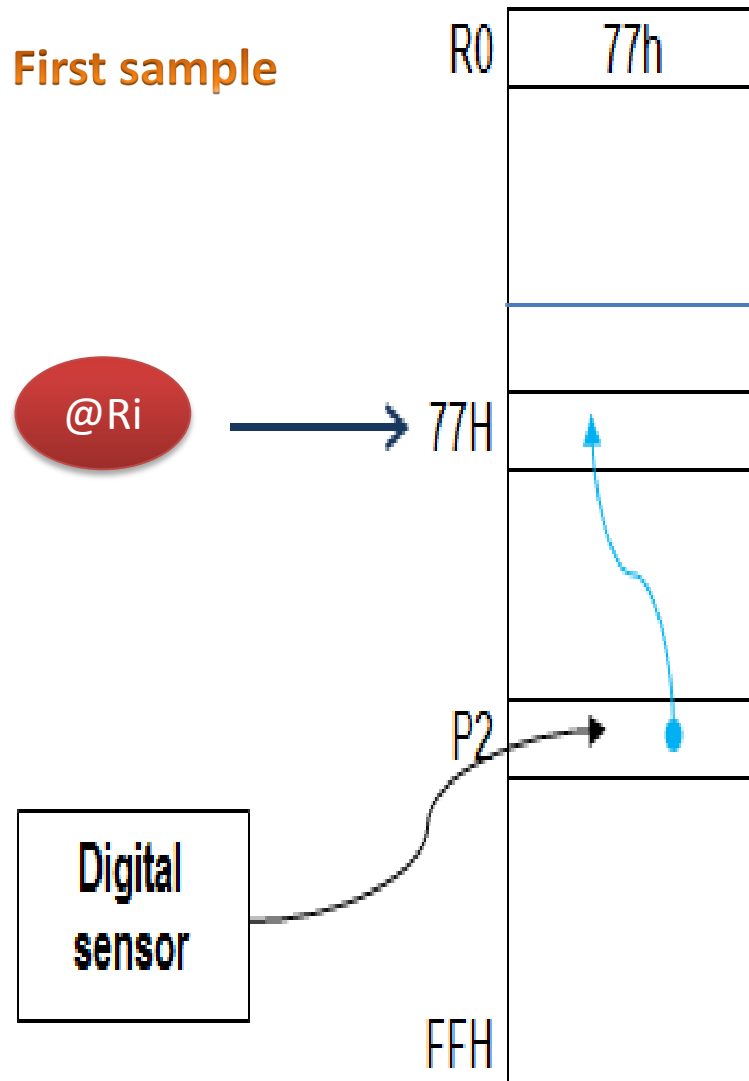
```
MOV @R0, P2 ; Moving 1st data to R77
```

```
INC R0 ; incrementing R0
```

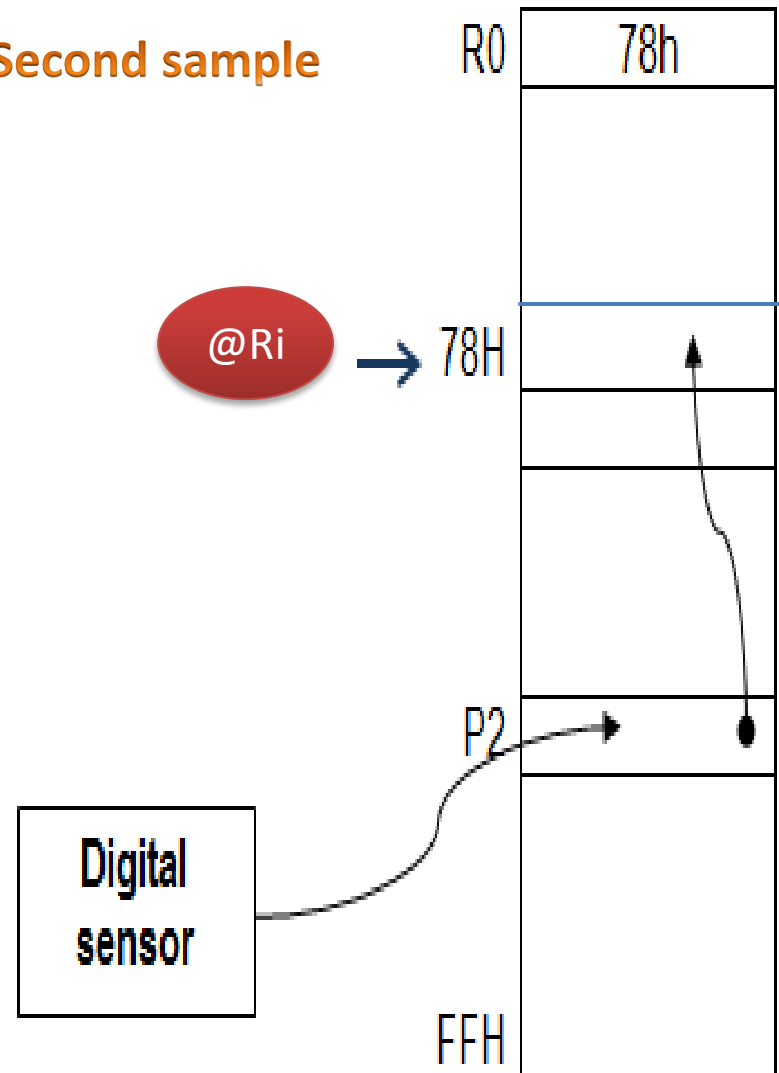
→

```
MOV @R0, P2 ; Moving 2nd data to R78
```

First sample



Second sample



The 16-bit Data transfer

Example → **MOV DPTR, #EE44h**

